GUÍA 2 PARA DESARROLLAR EN PLANTUML

Relación entre casos de uso, diagrama de clases de análisis y diagramas de secuencia de análisis

Caso base: SIMP – Sistema de Inventario de Materia Prima

Elemento	Descripción
Asignatura	Análisis y Diseño de Software
Tema	Trazabilidad entre casos de uso, clases de análisis y secuencias de análisis con PlantUML
Integrantes	Gabriel Báez Isaac Erazo Erika Guayanay
Caso base	SIMP – Sistema de Inventario de Materia Prima
Referencia base	01_Guia_PlantUML_CU_SIMP
Propósito	Guiar al estudiante para pasar del análisis funcional al análisis estructural y dinámico, manteniendo coherencia entre requisito, caso de uso, clase y secuencia.

1. Propósito de la guía

Esta guía complementa la guía inicial de casos de uso. Su finalidad es mostrar cómo un CRUD de Estudiante puede analizarse de forma progresiva mediante tres modelos conectados: casos de uso de análisis, clases de análisis y diagramas de secuencia de análisis.

El énfasis no está en programar la interfaz, sino en comprender qué responsabilidad cumple cada elemento del modelo y cómo se conserva la trazabilidad desde el requisito funcional hasta la interacción entre objetos.

2. Relación general entre los tres modelos

Modelo	Pregunta que responde	Producto en PlantUML	Uso en la siguiente etapa
Casos de uso de análisis	¿Qué necesita hacer el actor con el sistema?	Diagrama de casos de uso y matriz RF-CU.	Permite identificar responsabilidades, clases frontera, control y entidad.
Clases de análisis	¿Qué objetos participan para cumplir los casos de uso?	Diagrama de clases de análisis con estereotipos boundary, control, entity y repositorio.	Define los objetos que aparecerán en los diagramas de secuencia.
Secuencias de análisis	¿Cómo colaboran actor, frontera, control y entidad para ejecutar cada caso de uso?	Diagramas de secuencia por operación del CRUD del estudiante	Valida que el flujo sea coherente con los casos de uso y las clases identificadas.

3. Caso base actualizado: SIMP

El caso se basa en una interfaz sencilla denominada Sistema de Gestión de Materia Prima, con tres campos de entrada y cuatro operaciones principales.

Dato	Tipo sugerido	Descripción	Validación mínima
Producto	Texto	Información de artículos del inventario (Nombre, Categoría, Unidad de Medida, Stock Mínimo, Estado).	Nombre obligatorio, categoría y unidad de medida existentes, stock mínimo >= 0.

Unidad de Medida	Texto	Unidades asociadas a productos y compras (Nombre, Abreviatura, Estado).	Nombre y abreviatura obligatorios, sin duplicados.
Categoría	Texto	Clasificación para organización de stock y reportes (Nombre, Descripción, Estado).	Nombre obligatorio y único.
Consolidación	Texto	Cálculo de existencias integrando conteos físicos y compras.	Conteos físicos procesados y compras registradas previamente.

Botón / Acción de interfaz	Caso de uso asociado	Descripción de análisis
Gestionar Productos	CU001.1 Gestionar productos	Permite agregar, modificar, inactivar/eliminar y consultar productos en el catálogo.
Gestionar Unidades de Medida	CU001.2 Gestionar unidades de medida	Permite crear, actualizar, inactivar y listar unidades de medida disponibles.
Gestionar Categorías	CU001.3 Gestionar categorías de inventario	Administra la estructura de categorías de productos para la organización del stock.
Consolidar Inventario	CU001.4 Consolidar inventario del sistema	Ejecuta el cálculo automático de existencias integrando conteos físicos de cajeros y compras.

4. Requisitos funcionales de nivel 0

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe permitir la creación, modificación, consulta y eliminación de productos del inventario, asegurando que cada producto esté correctamente clasificado y medido.	Administrador	Gestionar productos
RF-02	El sistema debe permitir administrar las unidades de medida utilizadas para el registro de productos y compras.	Administrador	Gestionar unidades de medida
RF-03	El sistema debe permitir organizar los productos del inventario mediante categorías que faciliten la gestión y generación de reportes.	Administrador	Gestionar categorías de inventario
RF-04	El sistema debe permitir mantener actualizadas las existencias del inventario integrando los conteos físicos y las compras realizadas.	Administrador	Consolidar inventario del sistema

5. Diagrama de casos de uso de análisis

El diagrama de casos de uso representa las funciones visibles para el actor. En este caso, el Administrador académico interactúa con las operaciones CRUD y la validación se modela como comportamiento obligatorio para agregar y actualizar.

Código PlantUML sugerido

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador" as Admin
actor "Sistema" as Sistema
```

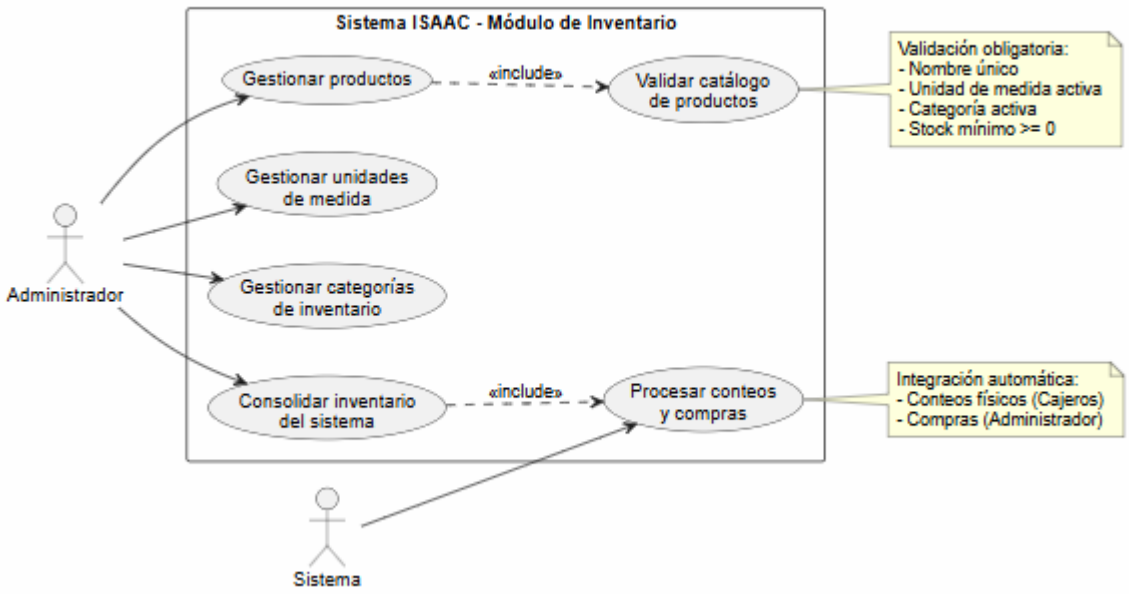
```
rectangle "Sistema ISAAC - Módulo de Inventario" {
  usecase "Gestionar productos" as UC1
  usecase "Gestionar unidades\nde medida" as UC2
  usecase "Gestionar categorías\nde inventario" as UC3
  usecase "Consolidar inventario\ndel sistema" as UC4
  usecase "Validar catálogo\nde productos" as UC5
  usecase "Procesar conteos\ny compras" as UC6
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4

UC1 ..> UC5 : <<include>>
UC4 ..> UC6 : <<include>>
Sistema --> UC6

note right of UC5
  Validación obligatoria:
  - Nombre único
  - Unidad de medida activa
  - Categoría activa
  - Stock mínimo >= 0
end note

note right of UC6
  Integración automática:
  - Conteos físicos (Cajeros)
  - Compras (Administrador)
end note
@enduml
```



6. Derivación de clases de análisis

A partir de los casos de uso seleccionados, identificamos las clases de análisis agrupadas por sus estereotipos correspondientes, manteniendo la separación de responsabilidades.

Tipo de clase	Responsabilidad en el análisis	Clase propuesta	Relación con el CRUD
Boundary / Frontera	Recibe los datos del actor y muestra los resultados de las operaciones en la interfaz.	`FormularioProducto`, `FormularioUnidadMedida`, `FormularioCategoria`, `PantallaConsolidacion`	Representan las pantallas e interfaces gráficas con campos de entrada (nombre, stock, categorías, etc.) y botones de acción para cada caso de uso.
Control	Coordina la lógica de negocio y el flujo de ejecución de cada caso de uso.	`ControlProducto`, `ControlUnidadMedida`, `ControlCategoria`, `ControlConsolidacion`	Validan los datos de entrada, gestionan las reglas de los formularios y coordinan la persistencia o el cálculo de existencias.
Entity / Entidad	Representa la información conceptual y estructural del dominio del negocio.	`Producto`, `UnidadMedida`, `Categoria`, `Inventario`	Contienen los atributos esenciales de los elementos del inventario de materia prima (nombre, stock mínimo, existencias actuales, etc.).
Repositorio / Colección	Conserva, recupera y actualiza los registros persistentes del sistema.	`RepositorioProducto`, `RepositorioUnidad`, `RepositorioCategoria`, `RepositorioInventario`	Permiten guardar, buscar, actualizar y recopilar los datos (incluyendo conteos físicos y compras) sin acoplarse directamente a la base de datos física.

Código PlantUML del diagrama de clases de análisis

```
@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
left to right direction

class "FormularioProducto" <<boundary>> {
```

```
  - txtNombre: String
  - cmbCategoria: String
  - cmbUnidadMedida: String
  - txtStockMinimo: int
  + clickGuardarProducto()
  + mostrarMensaje(msg: String)
}

class "FormularioUnidadMedida" <<boundary>> {
  - txtNombre: String
  - txtAbreviatura: String
  + clickGuardarUnidad()
}

class "FormularioCategoria" <<boundary>> {
  - txtNombre: String
  - txtDescripcion: String
  + clickGuardarCategoria()
}

class "PantallaConsolidacion" <<boundary>> {
  - fechaConsolidacion: Date
  + clickConsolidar()
  + mostrarResultado(detalles: String)
}

class "ControlProducto" <<control>> {
  + gestionarProducto(datos: Map): Resultado
}

class "ControlUnidadMedida" <<control>> {
  + gestionarUnidad(datos: Map): Resultado
}

class "ControlCategoria" <<control>> {
  + gestionarCategoria(datos: Map): Resultado
}

class "ControlConsolidacion" <<control>> {
  + consolidarInventario(fecha: Date): Resultado
}

class "Producto" <<entity>> {
  - nombre: String
  - stockMinimo: int
  - estado: boolean
}
```

```
class "UnidadMedida" <<entity>> {
  - nombre: String
  - abreviatura: String
}

class "Categoria" <<entity>> {
  - nombre: String
  - descripcion: String
}

class "Inventario" <<entity>> {
  - existenciasActuales: double
  - entradas: double
  - salidas: double
}

class "RepositorioProducto" <<repository>> {
  + guardar(p: Producto): void
  + buscarPorId(id: String): Producto
}

class "RepositorioUnidad" <<repository>> {
  + guardar(u: UnidadMedida): void
}

class "RepositorioCategoria" <<repository>> {
  + guardar(c: Categoria): void
}

class "RepositorioInventario" <<repository>> {
  + obtenerConteosYCompras(): List
  + actualizarExistencias(i: Inventario): void
}

FormularioProducto --> ControlProducto
FormularioUnidadMedida --> ControlUnidadMedida
FormularioCategoria --> ControlCategoria
PantallaConsolidacion --> ControlConsolidacion

ControlProducto --> Producto
ControlUnidadMedida --> UnidadMedida
ControlCategoria --> Categoria
ControlConsolidacion --> Inventario

ControlProducto --> RepositorioProducto
ControlUnidadMedida --> RepositorioUnidad
ControlCategoria --> RepositorioCategoria
ControlConsolidacion --> RepositorioInventario
```

```
@enduml
```

7. Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
Gestionar Productos	Captura nombre, categoría, unidad, stock mínimo y estado (FormularioProductos).	Valida datos de entrada y coordina la creación o actualización del producto.	Crea la entidad Producto y la amacena a través de RepositorioProducto	Secuencia: ingresar datos -> validar -> crear entidad -> guardar -> confirmar
Gestionar Unidades de Medida	Captura nombre y abreviatura de la unidad de medida.	Valida que los campos no estén vacíos y gestiona el registo.	Crea la entidad UnidadMedida y la persiste en RepositorioUnidad	Secuencia: ingresar datos-> validar -> crear entidad -> guardar -> confirmar.
Gestionar Categorias de Invetario	Captura nombre y descripción de la categoría.	Valida la información de la categoría y coordina su almacenamiento.	Crea la entidad Categoria y la persiste en RepositorioCategoria	Secuencia: ingresar datos -> validar -> guardar -> confirmar.
Consolidar Inventario del Sistema	Solicita la fecha y presenta el resultado de la consolidadción de existencias.	Recopila conteos físicos y compras, calculando las existencias finales	Lee registrosde RegistroInventario y actualiza las existencias integradas	Secuencia: solicitar consolidación -> recopilar datos -> calcular existencias -> actualizar -> mostrar.

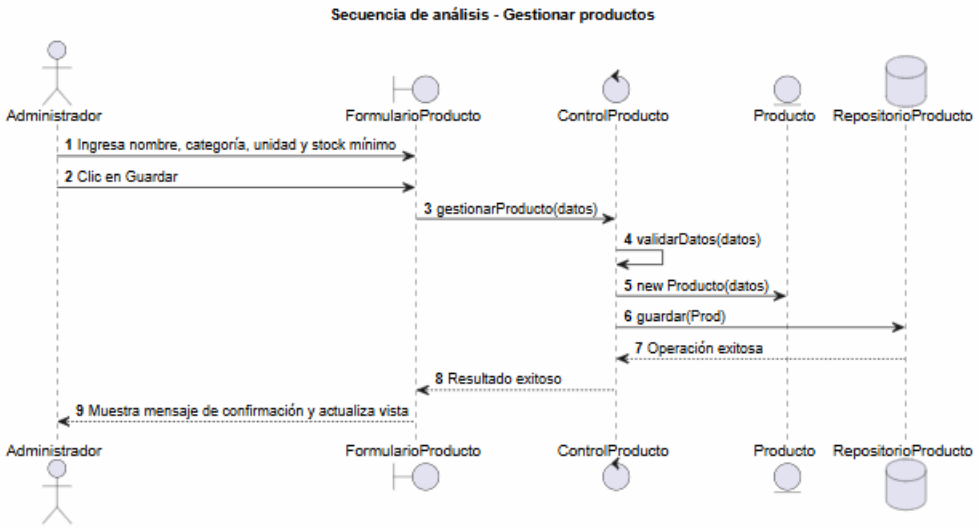
8. Diagramas de secuencia de análisis

Los diagramas de secuencia muestran cómo colaboran los objetos de análisis en el tiempo. Se recomienda elaborar un diagrama por cada caso de uso principal para evitar diagramas sobrecargados.

8.1 Secuencia: Gestionar Productos

```
@startuml
autonumber
title Secuencia de análisis - Gestionar productos
actor "Administrador" as Admin
boundary "FormularioProducto" as UI
control "ControlProducto" as Ctrl
entity "Producto" as Prod
database "RepositorioProducto" as Repo

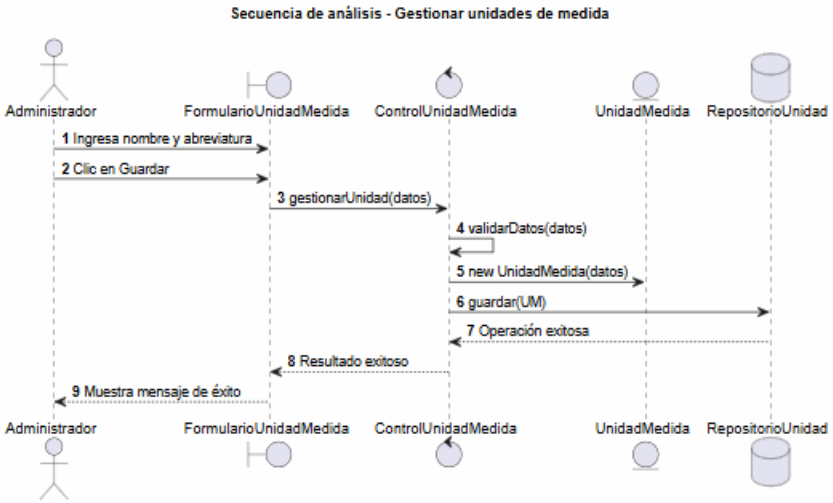
Admin -> UI : Ingresa nombre, categoría, unidad y stock mínimo
Admin -> UI : Clic en Guardar
UI -> Ctrl : gestionarProducto(datos)
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> Prod : new Producto(datos)
Ctrl -> Repo : guardar(Prod)
Repo --> Ctrl : Operación exitosa
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje de confirmación y actualiza vista
@enduml
```



8.2 Secuencia: Gestionar unidades de medida

```
@startuml
autonumber
title Secuencia de análisis - Gestionar unidades de medida
actor "Administrador" as Admin
boundary "FormularioUnidadMedida" as UI
control "ControlUnidadMedida" as Ctrl
entity "UnidadMedida" as UM
database "RepositorioUnidad" as Repo

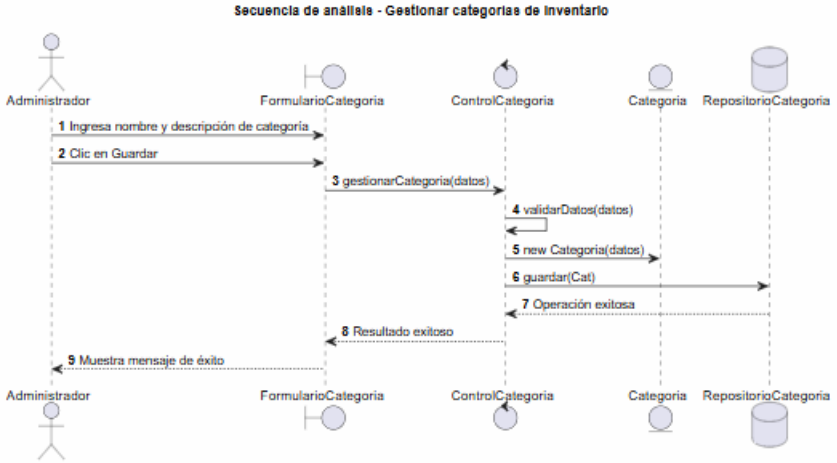
Admin -> UI : Ingresa nombre y abreviatura
Admin -> UI : Clic en Guardar
UI -> Ctrl : gestionarUnidad(datos)
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> UM : new UnidadMedida(datos)
Ctrl -> Repo : guardar(UM)
Repo --> Ctrl : Operación exitosa
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje de éxito
@enduml
```



8.3 Secuencia: Gestionar categor[ia de inventario

```
@startuml
autonumber
title Secuencia de análisis - Gestionar categorías de inventario
actor "Administrador" as Admin
boundary "FormularioCategoria" as UI
control "ControlCategoria" as Ctrl
entity "Categoria" as Cat
database "RepositorioCategoria" as Repo

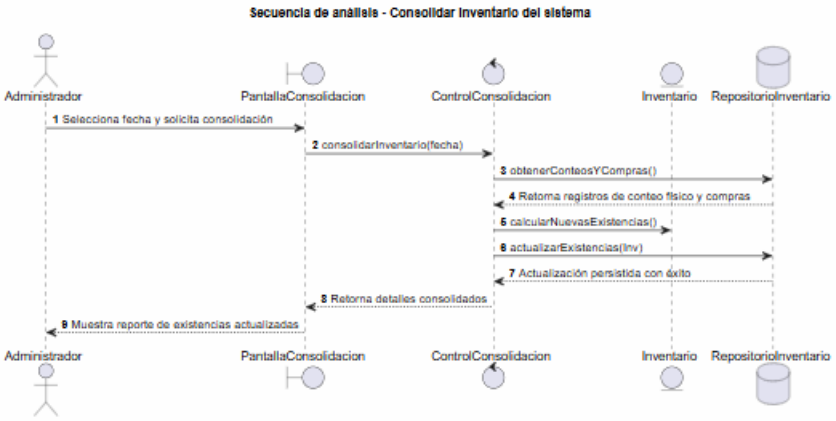
Admin -> UI : Ingresa nombre y descripción de categoría
Admin -> UI : Clic en Guardar
UI -> Ctrl : gestionarCategoria(datos)
Ctrl -> Ctrl : validarDatos(datos)
Ctrl -> Cat : new Categoria(datos)
Ctrl -> Repo : guardar(Cat)
Repo --> Ctrl : Operación exitosa
Ctrl --> UI : Resultado exitoso
UI --> Admin : Muestra mensaje de éxito
@enduml
```



8.4 Secuencia: Consolidar Inventario del Sistema

```
@startuml
autonumber
title Secuencia de análisis - Consolidar inventario del sistema
actor "Administrador" as Admin
boundary "PantallaConsolidacion" as UI
control "ControlConsolidacion" as Ctrl
entity "Inventario" as Inv
database "RepositorioInventario" as Repo

Admin -> UI : Selecciona fecha y solicita consolidación
UI -> Ctrl : consolidarInventario(fecha)
Ctrl -> Repo : obtenerConteosYCompras()
Repo --> Ctrl : Retorna registros de conteo físico y compras
Ctrl -> Inv : calcularNuevasExistencias()
Ctrl -> Repo : actualizarExistencias(Inv)
Repo --> Ctrl : Actualización persistida con éxito
Ctrl --> UI : Retorna detalles consolidados
UI --> Admin : Muestra reporte de existencias actualizadas
@enduml
```



9. Especificación breve de casos de uso conectada con las secuencias

Caso de uso	Precondición	Flujo principal resumido	Resultado esperado
Gestionar productos	El administrador debe estar autenticado y el nombre del producto no debe existir previamente en el sistema.	Ingresar datos del producto (nombre, categoría, unidad, stock mínimo) -> validar datos -> verificar duplicados -> guardar registro.	Producto registrado correctamente, disponible en el catálogo y listo para transacciones.
Gestionar unidades de medida	El administrador debe estar autenticado y la abreviatura/nombre de la unidad no debe estar registrada.	Ingresar nombre, abreviatura y estado -> validar datos requeridos -> verificar unicidad -> guardar unidad.	Unidad de medida creada y disponible para ser asignada a productos y compras.
Gestionar categorías de inventario	El administrador debe estar autenticado y la categoría no debe existir previamente.	Ingresar nombre de la categoría, descripción y estado -> validar campos -> guardar registro.	Categoría registrada exitosamente y disponible para organizar el inventario.

Consolidar inventario del sistema	El administrador debe estar autenticado y deben existir registros de compras o conteos físicos ingresados.	Solicitar consolidación -> el sistema recupera compras y conteos -> calcula existencias integrando ingresos y salidas -> actualiza stock.	Existencias del inventario actualizadas y visibles en la tabla principal con detalle de movimientos.

10. Procedimiento recomendado para el estudiante

- Revisar la interfaz CRUD e identificar campos de datos: ID, Nombre y Edad.
- Redactar requisitos funcionales de nivel 0 para Agregar, Actualizar, Eliminar y Mostrar Todo.
- Modelar los casos de uso en PlantUML y justificar únicamente las relaciones include necesarias.
- Derivar clases de análisis: una clase frontera para la interfaz, una clase control para coordinar la lógica, una entidad Estudiante y un repositorio o colección.
- Construir un diagrama de secuencia por cada caso de uso principal.
- Verificar que cada mensaje del diagrama de secuencia corresponda a una operación declarada en el diagrama de clases.
- Actualizar la matriz de trazabilidad para demostrar coherencia entre RF, CU, clases y secuencias.

11. Lista de verificación de coherencia

- Cada requisito funcional tiene al menos un caso de uso asociado.
- Cada caso de uso principal posee una secuencia de análisis propia.
- Las clases usadas en las secuencias existen en el diagrama de clases de análisis.
- Los mensajes de secuencia coinciden con operaciones de las clases de análisis.
- La entidad Estudiante conserva los atributos ID, Nombre y Edad.
- La validación de datos se aplica antes de agregar y actualizar.
- El diagrama no mezcla detalles de implementación de interfaz gráfica con responsabilidades de análisis.

12. Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad entre modelos	Relaciona correctamente RF, casos de uso, clases y secuencias.	5 pts	Matriz de relación completa.
Diagrama de casos de uso	Representa actor, sistema, casos de uso y relaciones include justificadas.	4 pts	Código y diagrama PlantUML.
Diagrama de clases de análisis	Identifica frontera, control, entidad y repositorio con responsabilidades claras.	4 pts	Código PlantUML de clases.
Diagramas de secuencia	Modela flujos coherentes para Agregar, Actualizar, Eliminar y Mostrar Todo.	5 pts	Cuatro secuencias PlantUML.
Presentación técnica	Documento ordenado, lenguaje claro y sin errores de sintaxis PlantUML.	2 pts	Entrega final documentada.

13. Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Casos de uso	Requisitos funcionales, diagrama CU y especificación breve.	Código PlantUML + captura.

Clases de análisis	Diagrama con boundary, control, entity y repositorio.	Código PlantUML + captura.
Secuencias de análisis	Una secuencia por cada operación CRUD principal.	Códigos PlantUML + capturas.
Trazabilidad	Matriz que conecte RF, CU, clases y secuencias.	Tabla de relación.
Conclusión técnica	Reflexión breve sobre la coherencia entre modelos.	Texto de cierre.

14. Conclusión técnica

En conclusión, se ha determinado que un sistema de software no debe concebirse como un conjunto de elementos aislados carentes de interrelación. En el contexto del proyecto SIMP, los componentes se integran de manera lógica y estructurada. Esto se evidencia en los casos de uso, los cuales especifican con precisión las necesidades y requerimientos del usuario.

Por su parte, las clases de análisis facilitan la arquitectura interna del sistema, permitiendo una clara separación de responsabilidades entre la interfaz visual (frontera), la lógica de procesamiento (control) y el almacenamiento de la información en la base de datos (entidad y repositorio).

Asimismo, preservar la trazabilidad garantiza la cobertura total de los requerimientos críticos, mitigando el riesgo de omitir funcionalidades esenciales o de implementar características no solicitadas por el cliente. En síntesis, este modelo metodológico proporciona una hoja de ruta estructurada que guía la transición desde la especificación documental hacia el diseño y desarrollo del software, asegurando la coherencia operativa durante todo el proyecto.

15. Fuente interna de referencia

- 01_Guia_PlantUML_CU_Analisis_SIMP
- Guía Version2 del CRUD Estudiante con campos ID, Nombre y Edad, adaptada a la interfaz de referencia proporcionada por la docente.

16. Anexos

Link Video: https://youtu.be/1eTiAOcUKBM